Chapter 4: Training a neural network model

In this chapter, you'll learn how to update spaCy's statistical models to customize them for your use case – for example, to predict a new entity type in online comments. You'll train your own model from scratch, and understand the basics of how training works, along with tips and tricks that can make your custom NLP projects more successful.

1 Training and updating models

2 Training and evaluation data

3 Creating training data (1)

4 Creating training data (2)

5 Configuring and running the training

6 The training config

7 Generating a config file

8 Using the training CLI

9 Exploring the model

10 Training best practices

11 Good data vs. bad data

12 Training multiple labels

13 Wrapping up

« Previous Chapter

3 Creating training data (1)

spaCy's rule-based `Matcher` is a great way to quickly create training data for named entity models. A list of sentences is available as the variable `TEXTS`. You can print it to inspect it. We want to find all mentions of different iPhone models, so we can create training data to teach a model to recognize them as `"GADGET"`.

- Write a pattern for two tokens whose lowercase forms match `"iphone"` and `"x"`.
- Write a pattern for two tokens: one token whose lowercase form matches `"iphone"` and a digit.

INPUT

```
import json
import spacy
from spacy.matcher import Matcher
from spacy.tokens import Span

with open("exercises/en/iphone.json", encoding="utf8") as f:
    TEXTS = json.loads(f.read())

nlp = spacy.blank("en")
matcher = Matcher(nlp.vocab)

# Two tokens whose lowercase forms match "iphone" and "x"
pattern1 = [{"LOWER": "iphone"}, {"LOWER": "x"}]

# Token whose lowercase form matches "iphone" and a digit
pattern2 = [{"LOWER": "iphone"}, {"IS_DIGIT": True}]

# Add patterns to the matcher and create docs with matched entities
matcher.add("GADGET", [pattern1, pattern2])
docs = []
for doc in nlp.pipe(TEXTS):
    matches = matcher(doc)
    spans = [Span(doc, start, end, label=match_id) for match_id, start, end in matches]
    print(spans)
    doc.ents = spans
    docs.append(doc)
```

Run code Submit

OUTPUT

```
[iPhone X]
[iPhone X]
[iPhone X]
[iPhone 8]
[iPhone 11, iPhone 8]
[]
```

4 Creating training data (2)

After creating the data for our corpus, we need to save it out to a `.spacy` file. The code from the previous example is already available.

- Instantiate the `DocBin` with the list of `docs`.
- Save the `DocBin` to a file called `train.spacy`.

INPUT

```
import json
import spacy
from spacy.matcher import Matcher
from spacy.tokens import Span, DocBin

with open("exercises/en/iphone.json", encoding="utf8") as f:
    TEXTS = json.loads(f.read())

nlp = spacy.blank("en")
matcher = Matcher(nlp.vocab)
# Add patterns to the matcher
pattern1 = ([{"LOWER": "iphone"}, {"LOWER": "x"}])
pattern2 = [{"LOWER": "iphone"}, {"IS_DIGIT": True}]
matcher.add("GADGET", [pattern1, pattern2])
docs = []
for doc in nlp.pipe(TEXTS):
    matches = matcher(doc)
    spans = [Span(doc, start, end, label=match_id) for match_id, start, end in matches]
    doc.ents = spans
    docs.append(doc)

doc_bin = DocBin(docs=docs)
doc_bin.to_disk("./train.spacy")
```

Run code Submit

6 The training config

The `config.cfg` file is the "single source of truth" for training a pipeline with spaCy. Which of the following is **not true** about the config?

- ◯ It allows you to configure the training process and hyperparameters.
- ◯ It helps make your training more reproducible.
- ◉ It creates an installable Python package with your pipeline.
- ◯ It defines the pipeline's components and their settings.

Submit

That's correct! The config file includes all settings related to training and how to set up the pipeline, but it doesn't package your pipeline. To create an installable Python package, you can use the `spacy package` command.

Mark as completed Next

11 Good data vs. bad data

Here's an excerpt from a training set that labels the entity type `TOURIST_DESTINATION` in traveler reviews.

```
doc1 = nlp("i went to amsterdem last year and the canals were beautiful")
doc1.ents = [Span(doc1, 3, 4, label="TOURIST_DESTINATION")]

doc2 = nlp("You should visit Paris once, but the Eiffel Tower is kinda boring")
doc2.ents = [Span(doc2, 3, 4, label="TOURIST_DESTINATION")]

doc3 = nlp("There's also a Paris in Arkansas, lol")
doc3.ents = []

doc4 = nlp("Berlin is perfect for summer holiday: great nightlife and cheap beer!")
doc4.ents = [Span(doc4, 0, 1, label="TOURIST_DESTINATION")]
```

Part 1

Why is this data and label scheme problematic?

- (●) Whether a place is a tourist destination is a subjective judgement and not a definitive category. It will be very difficult for the entity recognizer to learn.
- ( ) Paris should also be labelled as tourist destinations for consistency. Otherwise, the model will be confused.
- ( ) Rare out-of-vocabulary words like the misspelled 'amsterdem' shouldn't be labelled as entities.

Submit

That's correct! A much better approach would be to only label `"GPE"` (geopolitical entity) or `"LOCATION"` and then use a rule-based system to determine whether the entity is a tourist destination in this context. For example, you could resolve the entities types back to a knowledge base or look them up in a travel wiki.

Part 2

- Rewrite the `doc.ents` to only use spans of the label `"GPE"` (cities, states, countries) instead of `"TOURIST_DESTINATION"`.
- Don't forget to add spans for the `"GPE"` entities that weren't labeled in the old data.

INPUT

```
import spacy
from spacy.tokens import Span

nlp = spacy.blank("en")

doc1 = nlp("i went to amsterdem last year and the canals were beautiful")
doc1.ents = [Span(doc1, 3, 4, label="GPE")]

doc2 = nlp("You should visit Paris once, but the Eiffel Tower is kinda boring")
doc2.ents = [Span(doc2, 3, 4, label="GPE")]

doc3 = nlp("There's also a Paris in Arkansas, lol")
doc3.ents = [Span(doc3, 4, 5, label="GPE"), Span(doc3, 6, 7, label="GPE")]

doc4 = nlp("Berlin is perfect for summer holiday: great nightlife and cheap beer!")
doc4.ents = [Span(doc4, 0, 1, label="GPE")]
```

Run code Submit